華中科技大學

本科生毕业设计（论文）参考文献译文

译文出处： *Lu M, Jin P, Wang X, et al. ZoneKV: A Space-Efficient Key-Value Store for ZNS SSDs.60th ACM/IEEE Design Automation Conference,2023,2023: 1-6.*

院　　系	计算机科学与技术
专业班级	本硕博 2001 班
姓　　名	柳子淇
学　　号	U202015628
指导教师	刘渝

2024 年 3 月

译文要求

一、 译文内容须与课题（或专业内容）联系，并需在封面注明详细出处。

二、 出处格式为

图书：作者.书名.版本（第×版）.译者.出版地：出版者，出版年.起页～止页

期刊：作者.文章名称.期刊名称，年号，卷号（期号）：起页～止页

三、 译文不少于 5000 汉字（或 2 万印刷符）。

四、 翻译内容用五号宋体字编辑，采用 A4 号纸双面打印，封面与封底采用浅蓝色封面纸（卡纸）打印。要求内容明确，语句通顺。

五、 译文及其参考文献原文一起装订，顺序依次为封面、译文、文献。

六、 翻译应在第七学期完成。

译文评阅

导师评语

应根据学校“译文要求”，对学生译文翻译的准确性、翻译数量以及译文的文字表述情况等做具体的评价后，再评分。

评分：_____（百分制）　　指导教师（签名）：_____

2023 年 2 月 21 日

题目： ZoneKV：用于 ZNS 固态硬盘的空间高效的键值存储

摘要：在本文中，我们为 ZNS（分区命名空间）SSD 提出了一种新的节省空间的键值存储，称为 ZoneKV。我们观察到现有的将 RocksDB 适配到 ZNS SSD 的工作将导致区域碎片和严重的空间放大。因此，我们提出了一种基于生命周期的区域存储模型和特定于级别的区域分配算法，以在同一区域中存储具有相似生命周期的 SSTable。我们在真实的 ZNS SSD 上评估 ZoneKV。结果表明，与竞争对手使用 ZenFS 的 RocksDB 相比，ZoneKV 可以减少高达 60% 的空间放大，并保持更高的吞吐量。

索引术语——ZNS SSD、键值存储、空间效率

1 引言

ZNS SSD（分区命名空间 SSD）[1][2]是一种采用新型 NVMe 存储接口的新型 SSD。 ZNS SSD 将逻辑块地址 (LBA) 划分为多个相同大小的区域。每个区域只能顺序写入，复位后才能再次使用。由于强制按顺序写入区域，ZNS SSD 可以执行更粗粒度的地址转换（例如，将区域映射到闪存块），而不会牺牲性能。这种粗粒度映射允许在 ZNS SSD 内使用更小的 DRAM，从而显着节省成本[3]。此外，ZNS SSD 将垃圾收集和数据放置等功能留给主机端[1][4][5][6]，这使得用户能够设计有效的技术来利用 ZNS SSD 的优势，例如新型索引[7]和缓冲区管理器[8]。

ZNS SSD 与日志结构合并树（LSM 树）[9][10] 有着天然的契合。LSM-tree 将随机写入转换为顺序写入，大大提高了写入性能。它已被用于许多键值存储中，例如 BigTable、LevelDB 和 RocksDB。基于 LSM 树的键值存储包含一个内存组件，它由 MemTable 和 Immutable MemTable 组成。 MemTable 以仅追加模式存储最新的键值。当达到存储阈值时，Memtable 将转换为只读的 Immutable MemTable。基于 LSM 树的键值存储也由磁盘或 SSD 中的多个级别组成，每个级别由多个排序字符串表（SSTables）组成[11][12]。每个级别都有大小限制，并且大小限制呈指数增长。当级别超过其大小限制时，将触发压缩操作，通过将 L i 中的 SSTable 合并到 Li +1 来将数据下推。

最近，ZenFS [1]被提出以使 RocksDB 适应 ZNS SSD。据我们所知，到目前为止，ZenFS 是唯一支持 ZNS SSD 的基于 LSM 树的键值存储。 ZenFS 是西部数据公司开发的存储后端模块。它可以为新生成的 SSTable 分配区域，并从包含无效

数据的区域回收可用空间。ZenFS 采用经验方法将 SSTable 放入区域中，由于区域中的 SSTable 可能具有不同的生命周期，因此区域内会产生许多无效数据。例如，当选择 L 中的 SSTable 来执行压缩并且 L3 中的所有 SSTable 都应该存储在 Zone 1 中时，压缩后新生成的 SSTable 可以被写入到不同的 Zone，例如 Zone 2 。因此，ZenFS 将生成包含具有不同生命周期的 SSTable 的区域。这样一来，zone 中的无效数据就不会被及时回收，增加了 LSM-tree 的空间放大。

为了解决上述问题，在本文中，我们提出了一种用于 ZNS SSD 的新的节省空间的键值存储，称为 ZoneKV。ZoneKV 采用基于生命周期的区域存储模型和特定级别的区域分配算法，使每个区域中的 SSTable 具有相似的生命周期，从而比 ZenFS 具有更低的空间放大、更好的空间效率和更高的时间性能。简而言之，我们在本文中做出了以下贡献。

• 为了解决 ZNS SSD 上 LSM-tree 的空间放大问题，ZoneKV 提出了一种基于生命周期的区域存储模型，在一个区域中维护具有相似生命周期的 SSTable，以减少空间放大并提高空间效率。

• ZoneKV 提出了一种特定于级别的区域分配算法来组织 ZNS SSD 中 LSM 树的级别。特别是，ZoneKV 将 L 0 和 L 1 放在一个区域中，因为这两个级别中的 SSTable 具有相似的生命周期。另外，ZoneKV 将 Li (i ≥ 2)中的所有 SSTable 水平划分为切片，不同的切片存储在不同的 zone 中，使得每个 zone 的 SSTable 具有相同的生命周期。

• 我们将 ZoneKV 与 RocksDB [12] 以及真实 ZNS SSD 上最先进的系统 ZenFS [1] 进行比较。与 RocksDB 相比，ZoneKV 可以减轻约 50%-60%的空间放大，与 ZenFS 相比，这一改进约为 40%-50%。此外，ZoneKV 实现了最高的吞吐量。

2 相关工作

在传统的固态硬盘（SSD）中，设备对主机不是透明的。因此，主机无法执行应用程序感知的数据放置或其他操作以减少写放大和空间放大[13]。此外，设备端的垃圾回收（GC）使得 SSD 的性能不可预测[14]。ZNS SSD 可以被视为开放通道 SSD 的改进版本。ZNS 符合 ZAC/ZBC 规范[1][3]。ZNS SSD 支持完整的存储堆栈，包括底层块驱动到上层文件系统。此外，在 ZNS SSD 中，设备端不执行 GC，

这意味着主机必须执行主机级 GC。为了尽量减少主机级的 GC 开销，提出了一种新的 ZNS 接口 ZNS+[15]，以支持存储内部的区域压缩。专注于 ZNS SSD 上压缩的其他方法包括 CAZA[16]和 LL-压缩[17]。然而，这两种方法都会使得压缩后的 SSTables 散布在不同的区域中。从概念上讲，由同一压缩操作产生的 SSTables 应该具有相同的生命周期[5]。将它们放置在不同的区域中，对空间效率不友好。

到目前为止，据我们所知，ZenFS [1] 是唯一将 RocksDB 适配到 ZNS SSD 的键值存储。但 ZenFS 并没有充分考虑 SSTables 的生命周期，也没有完整的 GC 逻辑。因此，具有不同生命周期的 SSTables 可能会存储在同一区域中，导致严重的空间放大。在本文中，我们主要关注改进 ZenFS，并将 ZenFS 视为主要竞争对手。与 ZenFS 不同，我们建议将具有相似生命周期的 SSTable 存储在同一区域内，以减少 LSM 树的空间放大。

3 ZoneKV 的设计

在本节中，我们详细介绍 ZoneKV 的理念和设计。首先，我们分析现有方法的局限性。然后，我们解释一下 ZoneKV 的主要思想。在介绍了 ZoneKV 的架构之后，我们描述了 ZoneKV 中的关键技术，包括基于生命周期的区域存储和特定于级别的区域分配。

3.1 现有方法的局限性

将 RocksDB 移植到 ZNS SSD 的基本方法是让 RocksDB 将 SSTable 存储到特定的区域，这可以通过文件操作接口来实现[1]。虽然这种简单的实现可以支持 ZNS SSD 上的读/写操作，但它的空间利用率不高。这是因为 SSTable 随机分散在区域之间。由于每个 zone 中的 SSTable 可能有不同的生命周期，例如，一些 SSTable 经常更新，而另一些则从不更新，我们可以推断每个 zone 中会有很多无效数据，因为 ZNS SSD 必须使旧数据失效并保存写入新数据受影响区域末端的数据。在执行区域重置操作之前，这些无效数据将不会被重用。然而，由于区域重置操作成本高昂（必须执行许多数据移动），因此 ZNS SSD 不会重置区域，直到该区域的空间使用达到限制。因此，如果所有 SSTables 都是随机存储在区域中的，那么每个区域中将会产生越来越多的无效数据，这会导致空间放大率高和空间浪费。ZenFS 提出了一种改进的方法来在区域中存储 SSTables。它为每个活

跃区域维护最长的生命周期；如果新生成的 SSTable 的生命周期比活跃区域中的最长生命周期短，那么 SSTable 将被放入该区域中。与原始的 RocksDB 相比，ZenFS 可以减少空间放大，但具有不同生命周期的 SSTables 可能仍然分布在同一个区域中。请注意，具有不同生命周期的 SSTables 具有不同的更新频率。因此，一个区域内的某些 SSTables 可能已经被更新，而其他 SSTables 则没有，这浪费了区域容量，且空间效率不高。

3.2 ZoneKV 的思想

在本文中，我们将 SSTable 的生命周期定义为它的创建时间和失效时间之间的间隔。假设 SSTable 在 t 1 创建，并在 t 2 压缩或更新；其生命周期定义为 | t 2 − t 1 | 。由于我们不希望具有不同生命周期的 SSTable 在不同区域之间分散，这会产生更多无效数据并增加空间放大，因此我们建议将所有具有相似生命周期的 SSTable 存储到同一区域中。

然而，获取 SSTable 的准确生命周期并不是一件容易的事。例如，当我们需要将 SSTable 写入 LSM 树中的某个级别时，我们只知道 SSTable 的创建时间。在 SSTable 被更新或压缩之前，我们不会知道它的失效时间。我们观察到较高级别（例如 L0 ）的 SSTable 具有较小的生命周期，因为它们更有可能因内存中的数据刷新而被压缩[18]。因此，我们不会显式维护每个 SSTable 的生命周期信息。相反，我们使用级别编号作为生命周期的指标。此外，由于 RocksDB 对 L0[12]采用分层压缩，这将使 L 0 中的所有 SSTable 失效，并将它们与 L 1 中的 SSTable 合并，并产生新的 SSTable，最终写入 L 1 。因此，我们可以推断 L 0 中的所有 SSTable 具有相同的生命周期。另一方面，由于 L0 中 SSTable 的键范围重叠，因此在对 L0 进行分层压缩时，更有可能读取 L 1 中的所有 SSTable 并与 L 0 合并[19]。因此，L1 中的几乎所有 SSTable 都会因 L 0 中触发的分层压缩而失效，这意味着 L 1 中的所有 SSTable 与 L 0 中的 SSTable 具有相似的生命周期。

此外，对于 L 2 和 L 3 等较低级别，RocksDB 使用 Leveling Compaction [12]，以循环方式选择 L i (i ≥ 2) 中的一个 SSTable，并将该 SSTable 与 L_i +1 中所有重叠的 SSTable 合并 。为此，我们可以看到具有小键的 SSTable 将首先被选择作为受害者进行压缩，这意味着它们具有相似的生命周期。因此，我们对 L i (i ≥

2) 中的 SSTables 进行水平分区，并将每个分区放在不同的区域中。通过这样的机制，我们可以确保每个区域中的所有 SSTable 具有相似的生命周期，这有望提供较高的空间效率。

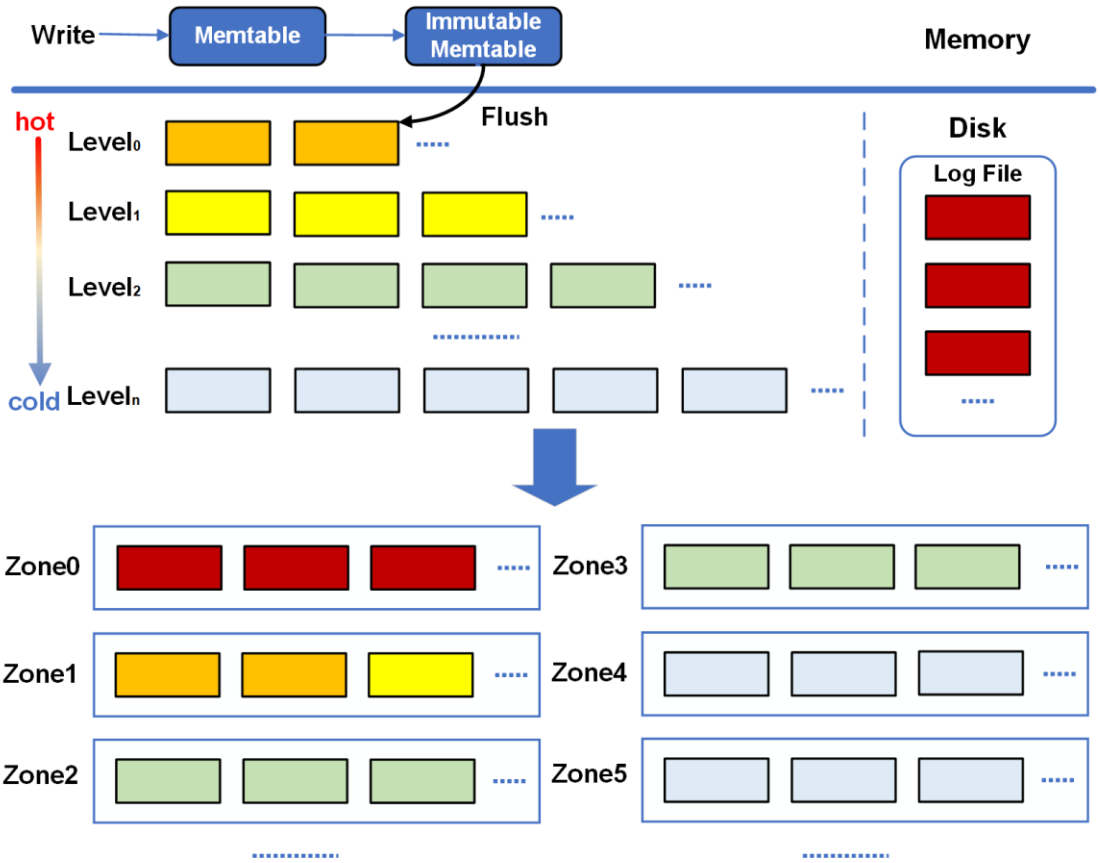


图 1　ZoneKV 的架构和数据布局

综上所述，ZoneKV 的思想可以概括为：

（1）基于生命周期的 Zone Storage。 ZoneKV 建议在一个区域中维护具有相似寿命的 SSTable，以减少空间放大并提高空间效率。 ZoneKV 不会显式维护每个 SSTable 的生命周期信息，而是使用每个 SSTable 的级别编号来隐式推断生命周期。这样的设计可以避免内存使用并维护生命周期信息的成本。

(2) 特定级别的区域分配。首先，ZoneKV 建议将 L 0 和 L 1 放在一个区域中，因为这两个级别中的 SSTable 具有相似的生命周期。其次，ZoneKV 将 L i (i ≥ 2) 中的所有 SSTable 水平划分为切片，每个切片存储在一个 zone 中。

3.3 ZoneKV 的架构

图 1 显示了 ZoneKV 的架构和数据布局。与 RocksDB 一样，ZoneKV 也包含内存组件和持久组件（即 ZNS SSD）。内存组件的作用与 RocksDB 相同，ZoneKV

的独特设计侧重于持久化组件，它由两个方面组成。首先，当 SSTables 通过刷新操作或压缩操作写入特定级别时，ZoneKV 根据生命周期信息将 SSTables 写入特定区域（选择合适区域的算法将在第 III-E 节中讨论） SSTables。其次，LSM 树中的每个级别都存储在不同的区域中，以使每个区域包含具有相似生命周期的 SSTable。如图 1 所示，ZoneKV 将日志文件放在单独的区域中，因为日志以仅追加的方式写入并且从不更新；因此它们的寿命可以被认为是无限的。ZoneKV 的实现是基于 RocksDB 的。与 ZenFS 一样，ZoneKV 修改了 RocksDB 中 FileSystemWrapper 类的接口，并通过 libzbd 直接与区域交互。传统的磁盘 I/O 堆栈需要内核文件系统、块层、I/O 调度层、块设备驱动层等一系列 I/O 子系统才能到达磁盘。这些长链总是会降低数据存储效率，间接降低磁盘吞吐量并增加请求延迟。 ZoneKV 针对 ZNS 进行了优化通过直接在 ZNS SSD 上执行端到端数据放置并绕过巨大的 I/O 堆栈。

3.4 基于生命周期的区域存储

如前所述，ZoneKV 建议在一个区域中维护具有相似生命周期的 STable，以减少空间放大。具体来说，我们将 L 0 和 L 1 中的 SSTable 放在一个 zone 中，将 L i (i ≥ 2) 中的 SSTable 放在多个 zone 中。对于每个 L i (i ≥ 2)，SSTable 根据其键范围进行分组，形成一组 SSTable 组。然后，我们将每个 SSTable 组放在一个区域中。这种划分有两个原因。首先，区域容量（例如，我们实验中的 1 GB）不足以存储低级别的所有 SSTable。其次，同一级别的 SSTable 具有不同的生命周期。特别是，在 RocksDB 中，具有小键的 SSTable 会被提前选择与其他 SSTable 合并。基于生命周期的 Zone 存储可以在同一个 Zone 或同一组 Zone 中存储具有相似生命周期的 SSTable。

算法 1：

Algorithm 1: Search for suitable zones

```
Function Find_zone(SSTable)
    lifetime_SSTable ← Get_lifetime(SSTable);
    zone ← Get_first_active_zone();
    while zone do
        if lifetime_SSTable = zone.lifetime_zone and
          Is_not_full(zone) then
            return zone;
        zone ← Get_next_active_zone();
    end
    if Get_num_empty_zone() > 0 then
        if Get_num_active_zone() <
          MAX_NUM_ACTIVE_ZONES then
            zone ← Assign_new_zone();
            return zone;
        else if Get_num_active_zone() =
          MAX_NUM_ACTIVE_ZONES then
            Close_active_zone();
            zone ← Assign_new_zone();
            return zone;
    else if Get_num_empty_zone() = 0 then
        zone ← Get_first_zone();
        while zone do
            if lifetime_SSTable < zone.lifetime_zone and
              Is_not_full(zone) then
                return zone;
            zone ← Get_next_zone();
        end
        zone ← Get_first_zone();
        while zone do
            if Is_not_full(zone) then
                return zone;
            zone ← Get_next_zone();
        end
    return NULL;
```

3.5 特定级别的区域分配

ZoneKV 采用特定级别的区域分配方案来实现基于生命周期的区域存储。为了避免内存成本并维持在 ZoneKV 中显式存储生命周期的开销，我们使用与级别相关的值来表示 SSTable 的生命周期。更具体地说，ZoneKV 将日志文件的生命周期标记为 - 1 ，这意味着日志文件具有无限的生命周期。接下来，我们将 L 0 和 L 1 中 SSTable 的生命周期设置为 1，这意味着这些 SSTable 的生命周期仅比日志文件的生命周期长。另外，L i (i ≥ 2) 中 SSTables 的生命周期被设置为 i 。数字越大意味着 SSTable 的生命周期越长，表明 SSTable 被更新或压缩的可能性

较小。

ZoneKV 根据特定级别的生命周期值将适当的区域分配给新生成的 SSTables 进行存储。算法 1 中描述了为 SSTable 找到合适区域的步骤。该算法的主要目标是从所有活动区域中找到与当前新生成的 SSTable 具有相同生命周期的区域。如果找到两个或多个区域，我们选择第一个找到的区域作为候选区域。如果在所有活动 zone 中都没有找到合适的 zone，则直接为 SSTable 分配一个新的 zone，并将该 zone 的生存期设置为 SSTable 的生存期值。如果没有找到合适的 zone，并且 ZNS SSD 中不存在空 zone，我们会寻找生命周期值大于 SSTable 的 zone。如果没有找到这样的 zone，那么任何有足够空间的 zone 都可以分配给当前的 SSTable。

4 性能评估

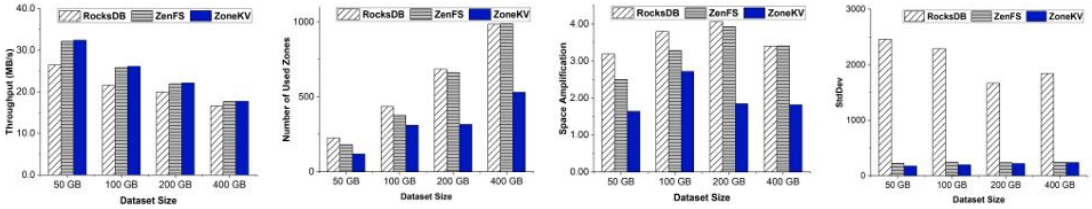
4.1 实验设置

所有实验均在配置了 Intel Xeon Gold 6348 CPU 2.60 GHz 和 256 GB DDR4 的 Linux (5.15.0) 上进行。我们的实验使用 RocksDB（7.8.0）提供的微基准（db_bench）。我们实验中的键都是 16 字节，值都是 1024 字节。对于 RocksDB，SSTable 的大小为 64MB，L 0 和 L 1 的阈值都是 256MB。我们在所有实验中均使用西部数据提供的 ZN540 ZNS SSD。 ZN540 的特性如表 1 所示。此外，我们将 ZoneKV 与 RocksDB 和 ZenFS 进行了比较。这里的 RocksDB 是指不做任何修改直接移植到 ZNS SSD 上的 RocksDB，不感知 ZNS 接口。 ZenFS [1]是唯一针对 ZNS SSD 优化 RocksDB 的键值存储。我们在实验中将 ZenFS 作为最先进的工作。

4.2 写入性能

在本实验中，我们使用 db_bench 中的 fillseq 和 fillrandom 工作负载。 fillseq 工作负载包含顺序写入，而 fillrandom 包含随机写入。在执行 fillrandom 之前，我们首先使用 fillseq 顺序写入 50 GB 键值数据以预热 ZNS SSD。

预热 ZNS SSD 后，我们使用 fillrandom 工作负载分别随机写入 50 GB、100 GB、200 GB 和 400 GB 键值数据。图 2 显示了随机写入性能的实验结果。



(a) Throughput. (b) Used zones. (c) Space amplification. (d) StdDev.

图 2 写入性能

从图 2（a）可以看出，在不同数据库大小的四种随机写入负载下，ZoneKV 的吞吐量明显高于 RocksDB，并且均略高于 ZenFS。当随机写入 50 GB 数据时，ZoneKV 的写入性能分别比 RocksDB 和带有 ZenFS 的 RocksDB 高 1 . 25 x 和 1.04 x。由于不同生命周期的 SSTable 可能分布在一个 zone 中，RocksDB 和 ZenFS 在重置时需要复制有效数据区域，占用部分写入带宽。导致 RocksDB 和 ZenFS 的写入性能随着运行而下降。 RocksDB 的性能比 ZenFS 差，因为每个 zone 的数据比较杂乱。

从图 2(b)可以看出，ZoneKV 使用的 zone 数量比 RocksDB 减少了 47.35%、28.28%、53.87%和 46.09%，比 RocksDB 减少了 34.25%、17.24%、52.27%和 46.26%。ZenFS 随机写入数据分别为 50GB、100GB、200GB、400GB 时的情况。大多数已使用的区域都是已满的区域，只有少数（大约十个）未满。

图 2(c)显示，在上述四种写负载下，ZoneKV 的空间放大比 RocksDB 小 48.59%、28.42%、54.43%和 46.47%，比 RocksDB 小 34.40%、17.33%、52.93%和 46.63 比 ZenFS 少 %。 ZoneKV 使用的 zone 数量比 RocksDB 和 ZenFS 少很多，ZoneKV 的空间放大也更低，说明 ZoneKV 比 RocksDB 和 ZenFS 更节省空间。

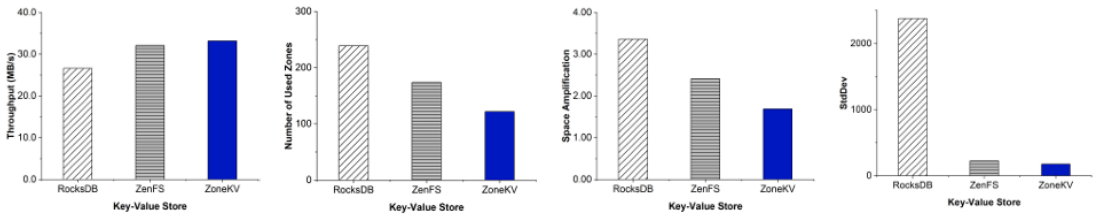
图 2(d) 显示了 ZoneKV 及其竞争对手的标准差 (StdDev)。ZoneKV 的 StdDev 比 RocksDB 和 ZenFS 更小，这表明 ZoneKV 的性能更稳定（更小的 StdDev 代表吞吐量更稳定）。这主要是因为 ZoneKV 在 Zone 重置过程中导致的数据迁移很少。

此外，如图 2（b）所示，当数据负载从 50GB 增长到 100GB，然后增长到 200GB，最后增长到 400GB 时，ZenFS 使用的区域数量猛增，甚至超过了 RocksDB。在这个过程中，ZoneKV 使用的 zone 数量缓慢增加，这说明 ZoneKV 具有更好的扩展性。由于 ZNS SSD 的容量有限，当写入键值存储的数据从 400GB 增加到特

定值时，ZenFS 和 RocksDB 已经空间不足，而 ZoneKV 仍然可以高性能运行。我们也在实验中证明了这一点。当我们尝试将 2 TB 键值数据写入 ZNS SSD 时，ZenFS 和 RocksDB 都失败，但 ZoneKV 仍然可以工作。

4.3 更新性能

在本实验中，我们使用 db_bench 中的覆盖工作负载来测试 ZoneKV 的随机更新性能。与随机写入实验一样，我们在覆盖数据之前随机写入 50 GB 数据来预热 ZNS SSD。再次将覆盖的数据大小设置为 50GB。



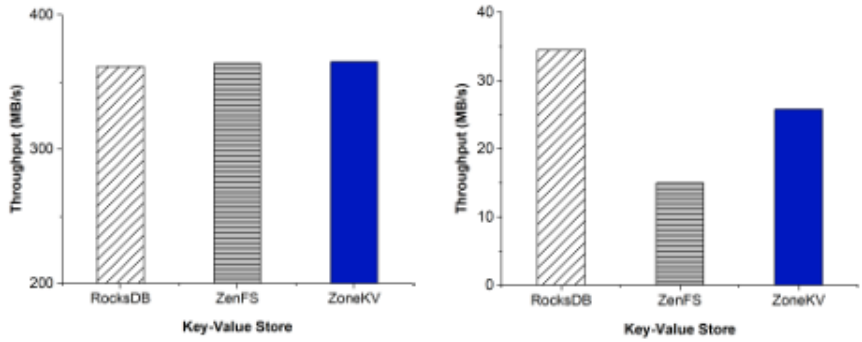
(a) Throughput. (b) Used zones. (c) Space amplification. (d) StdDev.

图 3 更新性能

如图 3 所示，ZoneKV 实现了最高的吞吐量。同时，它使用的区域数量最少并且空间放大率最低。此外，ZoneKV 的覆写性能也是最稳定的。这是因为 LSM-tree 中的更新是通过附加最新数据来完成的，而不考虑旧数据。因此，随机更新操作相当于随机写入操作；因此，更新性能与实验中的随机写入性能相似。

4.4 读性能

在本实验中，我们使用 db_bench 中的 readseq 和 readrandom 工作负载。readseq 工作负载包含顺序读取，而 readrandom 包含随机读取。我们首先使用 fillseq 顺序写入 50 GB 数据，然后使用 fillrandom 随机写入 50 GB 数据。接下来，我们通过覆盖更新 50 GB 数据，然后顺序读取 5 GB 数据并随机读取 5 GB 数据。上述所有操作均不间断地执行。



(a) Sequential read. (b) Random read.

图 4 读性能

如图 4 所示，所有三个键值存储展示出类似的顺序读性能，而 RocksDB 的表现略优于其他两个。ZoneKV 拥有第二高的随机读性能，而 ZenFS 的随机读性能不足。这是因为 ZenFS 在执行随机读之前有许多压缩操作，这将降低系统的吞吐量。相比之下，RocksDB 的压缩迅速完成，不会对随机读产生较大影响。此外，ZoneKV 引起的压缩次数少于 ZenFS，这有助于保持高随机读性能。

4.5 混合读/写性能

本实验中，我们使用 db_bench 中的 readrandomwriterandom 工作负载来测试混合读/写性能。 readrandomwriterandom 工作负载允许我们指定工作负载中的读写比率。在本实验中，我们采用 1:1 的读写比，即读取 5GB 数据，同时写入 5GB 数据。为了反映 ZoneKV 在其他指标方面的性能，我们报告了本实验中每个系统的延迟。

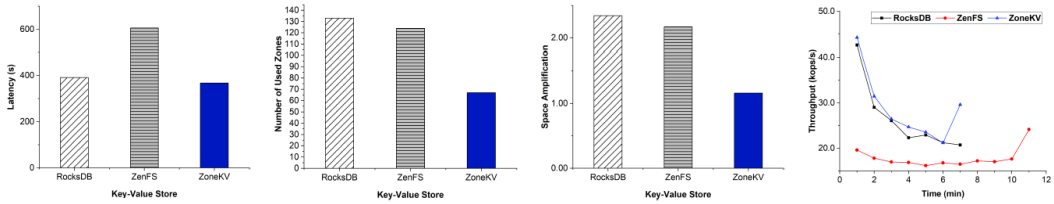
图 5(a) 显示 ZoneKV 实现了最低延迟。从图 5(b)和图 5(c)可以看出，ZoneKV 的区域数量最少，空间放大率最低。从图 5（d）中我们可以看到，ZoneKV 几乎一直具有最高的吞吐量，而 ZenFS 的吞吐量非常低。综上所述，ZoneKV 不仅节省空间，而且在读/写混合工作负载中具有高性能。

4.6 多线程性能

在这项实验中，我们测试了 ZoneKV 在多线程环境下的性能。我们使用 2、4、8 和 16 线程进行实验，默认采用随机写工作负载。实验前，我们先顺序写入 50GB 数据，然后使用多线程分别写入 50GB 数据。

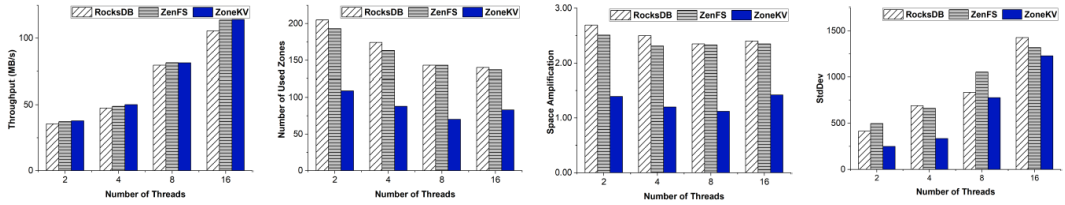
图 6 显示 ZoneKV 能够随线程扩展。在所有设置中，它总能保持最高的吞吐量。

此外，ZoneKV 还消耗了最少数量的区域，并实现了最低的空间放大。同时，ZoneKV 的标准差也小于 RocksDB 和 ZenFS。总的来说，ZoneKV 在多线程环境下的性能与单线程运行时一致，这意味着 ZoneKV 能在多线程设置中保持稳定且可扩展的性能。



(a) Latency. (b) Used zones. (c) Space amplification. (d) Operations per second.

图 5 混合读写性能



(a) Throughput. (b) Used zones. (c) Space amplification. (d) StdDev.

Fig. 6. Multi-thread performance.

图 6 多线程性能

5 总结

在本文中，我们提出了一种新的键值存储系统，专为 ZNS SSD 设计，称为 ZoneKV，旨在减轻空间放大效应并保持高性能。ZoneKV 的新颖之处在于基于生命周期的区域存储和特定级别的区域分配，这可以减少使用的区域数量，并降低 ZNS SSD 上 LSM 树的空间放大。我们通过实验证明，ZoneKV 在各种工作负载和设置下的性能超过了 RocksDB 和最新的 ZenFS。特别是，ZoneKV 可以在保持比竞争对手更高吞吐量的同时，将空间放大减少高达 60%。此外，多线程设置下的实验显示，ZoneKV 能在多线程环境下保持稳定且高性能。

6 主要参考文献

[1] M. Bjørling, A. Aghayev, et al., ZNS: avoiding the block interface tax for flash-based SSDs, Proceedings of the USENIX ATC, 2021, 689–703.

[2] H. Shin, M. Oh, et al., Exploring performance characteristics of ZNS SSDs: Observation and implication, Proceedings of the NVMSA, 2020, 1–5.

[3] T. Stavrinos, D. S. Berger, et al., Don’t be a blockhead: zoned namespaces make

work on conventional SSDs obsolete, Proceedings of the HotOS, 2021, 144–151.

[4] G. Choi, K. Lee, et al., A new LSM-style garbage collection scheme for ZNS SSDs, Proceedings of the HotStorage, 2020.

[5] H. Wang, Y. Liu, et al., Efficient data placement for zoned namespaces (ZNS) SSDs, Proceedings of the NPC, 2022, 302–314.

[6] H. Bae, J. Kim, et al., What you can't forget: exploiting parallelism for zoned namespaces, Proceedings of the HotStorage, 2022, 79–85.

[7] P. Jin, X. Zhuang, et al., Exploring index structures for zoned namespaces SSDs, Proceedings of the IEEE BigData, 2021, 5919–5922.

[8] Y. Lv, P. Jin, et al., Zonedstore: A concurrent zns-aware cache system for cloud data storage, Proceedings of the ICDCS, 2022, 1322–1325.

[9] P. E. O'Neil, E. Cheng, et al., The log-structured merge-tree (LSM-tree), Acta Informatica, vol. 33, no. 4, 1996, 351–385.

[10] D. R. Purandare, P. Wilcox, et al., Append is near: Log-based data management on ZNS SSDs, Proceedings of the CIDR, 2022.

[11] F. Chang, J. Dean, et al., Bigtable: A distributed storage system for structured data, ACM Transactions on Computer Systems, vol. 26, no. 2, 2008, 4:1–4:26.

[12] S. Dong, A. Kryczka, et al., Evolution of development priorities in key-value stores serving large-scale applications: The RocksDB experience, Proceedings of the FAST, 2021, 33–49.

[13] N. Agrawal, V. Prabhakaran, et al., Design tradeoffs for SSD performance, Proceedings of the USENIX ATC, 2008, 57–70.

[14] J. Kim, D. Lee, et al., Towards SLO complying SSDs through OPS isolation, Proceedings of the FAST, 2015, 183–189.

[15] K. Han, H. Gwak, et al., ZNS+: advanced zoned namespace interface for supporting in-storage zone compaction, Proceedings of the OSDI, 2021, 147–162.

[16] H. Lee, C. Lee, et al., Compaction-aware zone allocation for LSM based key-value store on ZNS SSDs, Proceedings of the HotStorage, 2022, 93–99.

[17] J. Jung, D. Shin, Lifetime-leveling LSM-tree compaction for ZNS SSD,

Proceedings of the HotStorage, 2022, 100–105.

[18] T. Yao, J. Wan, et al., Geardb: A GC-free key-value store on HMSMR drives with gear compaction, Proceedings of the FAST, 2019, 159–171.

[19] T. Yao, Y. Zhang, et al., Matrixkv: Reducing write stalls and write amplification in LSM-tree based KV stores with matrix container in NVM, Proceedings of the USENIX ATC, 2020, 17–31.